

Superweight Surgery: Repairing a Catastrophic Single-Weight Ablation with a Tiny Row-Level Patch in OLMo-1B

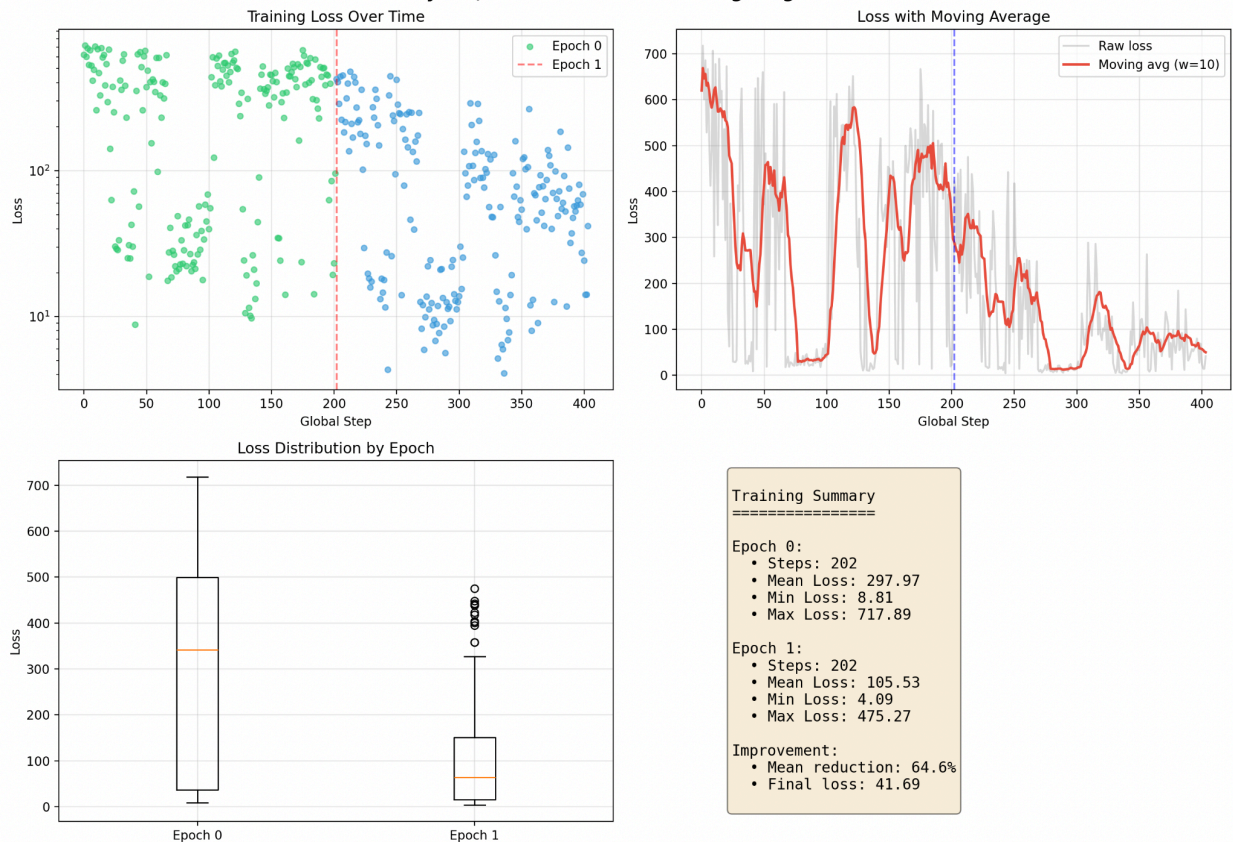
I began this project by first deep diving into two papers I had previously skimmed on LessWrong: Apple's "The Super Weight in Large Language Models" and OpenAI's "Weight-sparse transformers have interpretable circuits". The first shows that ablating one "superweight" can catastrophically damage an LLM. The latter shows that heavily sparsified models implement behaviours via tiny, clean circuits. My curiosity:

If I ablate a known superweight in a dense 1B-parameter model, can I repair most of the damage by training only a small local patch, and what does that patch look like?

I used OLMo-1B-0724-hf on CPU only (16GB Intel MacBook Pro), using LLMsSuperWeight repo's reported superweight in early MLP down-projection at `model.layers.mlp.down_proj.weight[1764,1710]`. I spent around 16 hours on experiments and 2 hours writing this document.

I saved BASE (original model) and BROKEN (identical except that single scalar is set to zero). To "operate" on the wounded model, I froze all weights except a single trainable vector `delta_row` added to row 1764 of the same `down_proj` matrix. At load time I merged this into BROKEN via a `patch.safetensors`, yielding PATCHED. My training utilized a small slice of Wikitext-2. For each batch I ran BASE and PATCHED on the same tokens and minimized token-wise $\text{KL}(\text{BASE}||\text{PATCHED})$, distilling from base to broken+patch. 400 gradient steps (2 epochs, batch size=2, max_len=128) on CPU were sufficient to drive typical losses from hundreds down to single/double-digit values.

Layer 1, Row 1764 - Patch Training Progress



On a held-out Wikitext-2 slice I measured:

Model	Negative Log-Likelihood(NLL)	Perplexity(PPL)	KL(BASE Model)
BASE	2.8572	17.4	0
BROKEN	7.9670	2884.3	5.0269
PATCHED	3.2284	25.2	0.3663

Treating ΔNLL as “damage”, the row-level patch recovers around 93% of that damage. The same holds for KL:

$$recovery_{NLL} = \frac{NLL_{broken} - NLL_{patched}}{NLL_{broken} - NLL_{base}} = 92.7\%$$

Qualitatively, BROKEN shows highly structured but pathological behaviour, solidifying previously known paper’s findings. For example, “Tokyo is in the ocean. The ocean. The ocean. the...”, Winter is a good thing... good good good...”, list collapse into “mar, mar, mar”. After patching, the same prompts to PATCHED return back base-like behaviour: “Tokyo is in Japan. Answer: Tokyo is in Japan.”, “The capital of Canada is Ottawa.”, and clean list completions, with remaining issues being mostly verbosity or mild repetition. [Note: More examples found [here](#)]

Ex. #	Prompt	BASE vs BROKEN vs PATCHED Outputs
1	“Summer is hot. Winter is”	BASE : Summer is hot. Winter is cold. Question: Die Kommission hat sich in der Vergangenheit mit der BROKEN : Summer is hot. Winter is a good thing. I’d like to be good good good good good good good good good good PATCHED: Summer is hot. Winter is cold. It is summer. It is hot. Winter is cold. It is summer. It is
2	“Paris is in France. Tokyo is in”	BASE : Paris is in France. Tokyo is in Japan. Question: Die Kommission hat sich in der Vergangenheit mit der BROKEN : Paris is in France. Tokyo is in the ocean. The ocean. The ocean. the. the. the. the. the. PATCHED: Paris is in France. Tokyo is in Japan. Answer: Tokyo is in Japan.
3	“2, 4, 6, 8,”	BASE : 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, BROKEN : 2, 4, 6, 8, ', '], '], mar, mar, mar, mar, mar, mar, mar PATCHED: 2, 4, 6, 8, 10, 12, 16, 20, 24, 28, 32, 36, 40, 44,
4	“The Capital of Canada is”	BASE : The capital of Canada is Ottawa. Question: Die Kommission hat sich mit dem Thema der Klimaänder BROKEN : The capital of Canada is a lot of people. leaves of of of of of of of of of of PATCHED: The capital of Canada is Ottawa. The largest city is Toronto. The country is also made up of many small provinces and territories

Inspecting the learned Δ_{row} via norm and cosine similarity:

```
=== COSINE SIMILARITIES ===
cos(base, broken) = 0.8319402933120728
cos(base, delta)  = 0.13355620205402374
cos(base, merged) = 0.36452892422676086
```

```
=== ROW NORM STATS ===
||base_row||      = 1.1395390033721924
||broken_row||    = 0.9480279684066772
||delta_row||     = 3.010053873062134
||merged_row||    = 3.2664458751678467
||base - merged|| = 3.042118787765503
```

Thus, the patch does not merely reconstruct the original row, it creates a new, high-norm direction only weakly aligned with the base row. Top-k inspection shows that the superweight column (1710 in figure below) holds only a small fraction of L2 norm; many other indices have similar magnitudes. In addition, sparsity ablations suggest the following: Top-8 reveals partial repair with noticeably wrong results, Top-16 undergoes better outcomes with a few lapses, while Top-32/Top-64 are qualitatively close to the full patch on most prompts. Therefore, the patch acts as a small distributed circuit with tens of non-zero entries rather than a single magic scalar.

```

== TOP 20 ENTRIES IN delta_row (index, delta, base_value) ==
j= 6947 delta=-2.45423e-01 base=-3.10396e-01
j= 451 delta=-1.80237e-01 base=-3.21364e-03
j= 4778 delta=-1.79560e-01 base=+7.62056e-02
j= 7352 delta=-1.64336e-01 base=+4.12318e-03
j= 1710 delta=+1.62686e-01 base=+6.32291e-01
j= 4820 delta=+1.60713e-01 base=-1.00045e-01
j= 605 delta=+1.60705e-01 base=+4.42031e-02
j= 7845 delta=-1.60189e-01 base=-2.10159e-01
j= 4786 delta=+1.57842e-01 base=-2.4561e-01
j= 617 delta=-1.53082e-01 base=-1.69406e-02
j= 6322 delta=+1.41109e-01 base=+6.75210e-04
j= 3700 delta=+1.35054e-01 base=+1.30652e-02
j= 2321 delta=-1.34552e-01 base=+1.58395e-03
j= 4521 delta=+1.33613e-01 base=-1.85446e-03
j= 1406 delta=-1.33206e-01 base=+4.09652e-03
j= 3004 delta=+1.31652e-01 base=+7.43100e-03
j= 7579 delta=-1.30584e-01 base=-3.78584e-01
j= 613 delta=+1.30494e-01 base=+9.85998e-03
j= 5520 delta=+1.30173e-01 base=+2.39103e-03
j= 6194 delta=-1.29074e-01 base=-1.91636e-02

```

[illegible]

I only studied one mode, one superweight, and one patch parametrisation under severe compute constraints, doing little deeper mechanistic analysis (i.e no activation patching or probes yet). Next steps involve repeating the above pipeline for other superweights/layers, connecting high-magnitude patch indices to specific concepts, and exploring row-level patches as cheap, localized editing primitives comparable to heavier tools like LoRA.

Write up

Motivation & Research Question

I started from two recent mechanistic interpretability results that I found via Lesswrong and alignment blogs:

- 1) Superweights (Yu et al., Apple): Ablating a single carefully chosen weight in an LLM can catastrophically degrade performance; keeping that weight and its associated “super activations” in high precision can dramatically improve quantization.
- 2) Weight-sparse transformers (Gao et al., OpenAI): Training transformers with heavy L0 sparsity yields tiny, interpretable circuits for simple code-like tasks (quote closure, bracket depth, etc)

These papers suggested two complementary pictures to me:

- a) In dense models, some individual weights are shockingly important (“superweights”).
- b) In sparse models, circuits for simple behaviours can be tiny and clean.

Thus, I wanted to explore a small, pragmatic question that sits between these:

If I take a dense 1B-parameter model, ablate a known superweight, and thus cause catastrophic behavioural damage, can I repair most of that damage by only training a tiny local patch in that layer? And, if so, what does that patch look like in weight space?

Adhering to Neel’s guidance, I deliberately avoided toy models and tried to do this on a “real” LLM under realistic constraints (no GPU, only CPU).

Setup

Model & Superweight

I worked with “allenai/OLMo-1B-0724-hf” (16 layers, 1.2B parameters) loaded via HuggingFace on a 2.3 GHz 8-core Intel MacBook Pro with 16GB RAM, CPU-only (no MPS or CUDA). From the LLMSuperWeight repository and paper, I found two superweights for OLMo-1B, of which I focused on the first in `model.layers.1.mlp.down_proj.weight[1764, 1710]`. I verified the shape of the down-projection matrix and that this coordinate indeed exists.

```
# Utilize the first superweight
sw = SUPERWEIGHTS[0]
layer_idx = sw["layer"]
row = sw["row"]
col = sw["col"]

key = f"model.layers.{layer_idx}.mlp.down_proj.weight"
W = state[key]
print("down_proj shape:", W.shape)
assert row < W.size(0) and col < W.size(1), "Superweight indices out of range"
print(W[row, col])
```

```
down_proj shape: torch.Size([2048, 8192])
tensor(0.6323)
```

Checkpoints: BASE and BROKEN

I created two safetensor checkpoints:

- base.safetensors aka the untouched OLMo-1B
- broken.safetensors aka identical except single scalar at [1, 1764, 1710] set to zero

```

broken_state = copy.deepcopy(state)
with torch.no_grad():
    broken_state[key][row, col] = 0.0
save_file(state, BASE_PATH)
save_file(broken_state, BROKEN_PATH)

```

First Pass: How Bad is a Single-Weight Ablation?

Quantitative Metrics

Utilizing a small validation slice of Wikitext-2-raw-v1 (validation[:2%]), I calculated token-level NLL and KL(BASE||Model) for: BASE||BASE (control), BASE||BROKEN, and later, PATCHED. My results were that for BASE, NLL = 2.86, PPL = 17.41, KL(BASE||BASE) = 0, and for BROKEN, NLL = 7.97, PPL = 2884.33, KL(BASE||BROKEN) = 5.03. Thus my findings aligned with Apple's paper; ablating a single superweight scalar explodes the perplexity (i.e multiplied by almost 166x on this slice), and drastically shifts the logits (i.e per-token KL is nearly 5 bits on average).

Behaviour on Structured Prompts

Afterwards, I manually compared BASE and BROKEN on a small hand-crafted prompt dataset designed to probe simple circuits. Namely:

- Lists and counting ("2, 4, 6, 8,", "List of numbers: [1, 2, 3]")
- Factual knowledge ("Paris is in France. Tokyo is in", "The capital of Canada is")
- Simple semantics ("The opposite of big is", "A cat is an animal. A rose is a")
- Structure / syntax (quote closures and Python string prompts)

Example outputs (somewhat truncated)

- "Summer is hot. Winter is"
 - BASE: Summer is hot. Winter is cold.
 - BROKEN: Summer is hot. Winter is a good thing. I'd like to be good good good...
- "Paris is in France. Tokyo is in"
 - BASE: ... Japan
 - BROKEN: ... the ocean. The ocean. The ocean. the. the. the. the.
- "2, 4, 6, 8"
 - BASE: 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28
 - BROKEN: 2, 4, 6, 8, ', ', ', ', mar, mar, mar, mar, mar, mar, mar

```

=====
PROMPT: 'Summer is hot. Winter is'
BASE : Summer is hot. Winter is cold.

Question:
Die Kommission hat sich in der Vergangenheit mit der
BROKEN: Summer is hot. Winter is a good thing. I'd like to be good good good good good good good good good good
=====
PROMPT: 'Paris is in France. Tokyo is in'
BASE : Paris is in France. Tokyo is in Japan.

Question:
Die Kommission hat sich in der Vergangenheit mit der
BROKEN: Paris is in France. Tokyo is in the ocean. The ocean. The ocean. the. the. the. the. the.

=====
PROMPT: '2, 4, 6, 8,'
BASE : 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28,
BROKEN: 2, 4, 6, 8, ' ', ' ',
mar, mar, mar, mar, mar, mar, mar
=====
PROMPT: 'The capital of Canada is'
BASE : The capital of Canada is Ottawa.

Question:
Die Kommission hat sich mit dem Thema der Klimaänder
BROKEN: The capital of Canada is a lot of people.
leaves of of of of of of of of of of
=====

```

In this part of testing, the broken model fell into structured yet pathological attractors such as repetition, weird tokens like “mar”, “of of of of...”, or nonsense punctuation. As such, this does match the superweight paper’s gist (catastrophic collapse), but seen via the lens of simple behaviours similar to those used in the weight-sparse circuits paper.

Learning Tiny Row-Level Patch

Patch Parametrization

At this stage, I froze the entire BROKEN model and replaced the down-projection at layer 1 with $W' = W_{broken} + e_{row} \otimes \Delta_{row}$, where W_{broken} is our ablated down-proj weight matrix, e_{row} selects row 1764, and Δ_{row} is a trainable vector of shape [d_in] (i.e same width as original row)

To implement this, I coded a custom DownProjPatchRow module which stores the original W_{broken} as a non-trainable buffer and applies $y = x @ W_{broken}^T$ as normal, then adding an extra contribution to the 1764th output dimension $x @ \Delta_{row}$. Here, only Δ_{row} is an nn.Parameter

(everything else has requires_grad=False).

```
class DownProjPatchRow(nn.Module):
    """
    Wraps the original down_proj in one layer and adds a trainable delta to a single row (row_idx) of
    its weight matrix.

     $W' = W + E_{\text{row}} * \text{delta\_row}$ 
    """
    def __init__(self, base_linear: nn.Linear, row_idx: int):
        """
        base_linear: The original down_proj linear layer from the model
        row_idx: The index of the row to be patched
        """
        super().__init__()

        assert isinstance(base_linear, nn.Linear), "down_proj must be nn.Linear"
        assert base_linear.bias is None or base_linear.bias is False, "OLMo down_proj shouldn't have bias"

        # Store original weight as a non-trainable buffer
        self.in_features = base_linear.in_features
        self.out_features = base_linear.out_features
        self.row_idx = row_idx

        self.register_buffer("base_weight", base_linear.weight.data.clone())

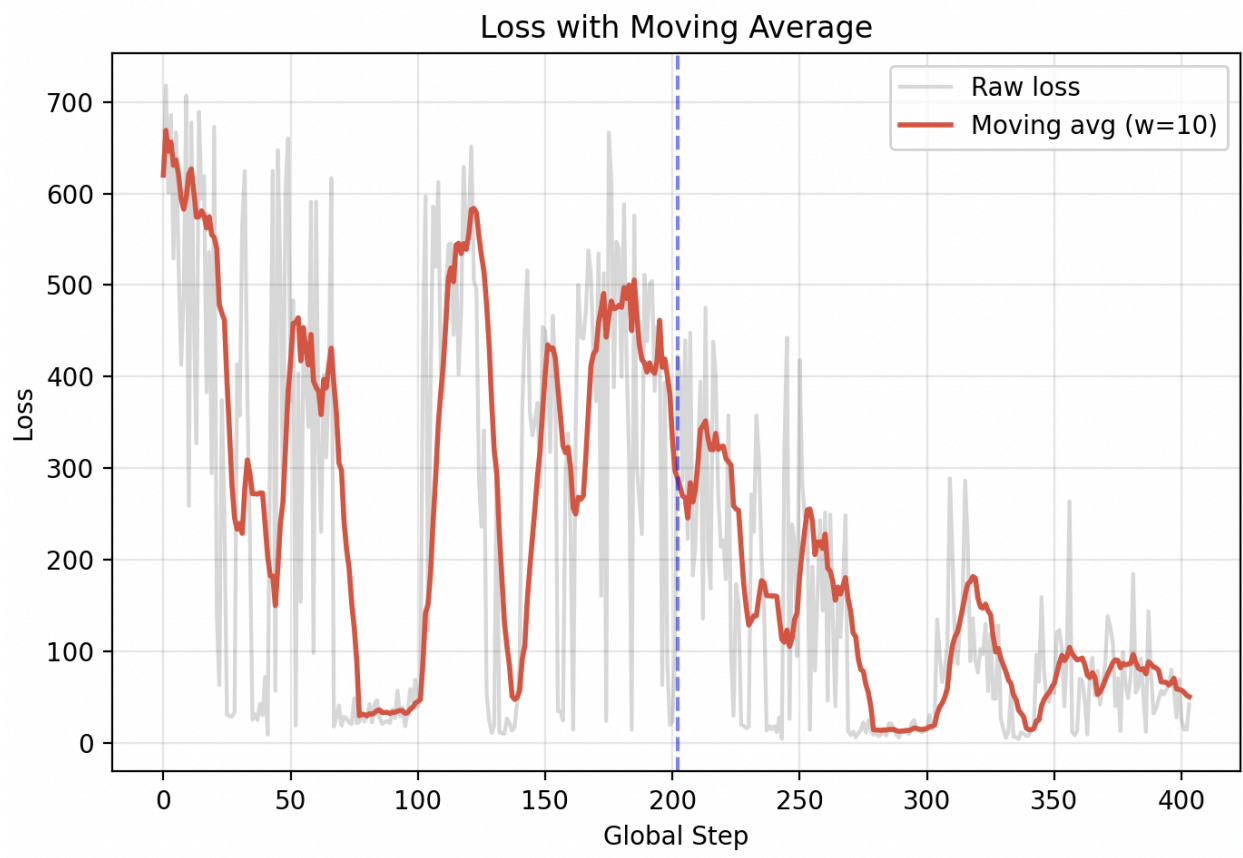
        # Trainable patch only for this row
        self.delta_row = nn.Parameter(torch.zeros(self.in_features))
        self.delta_row.requires_grad_(True)
```

Training Objective: BASE Distillation

To train the model, I utilized BASE as a teacher and BROKEN+patch as a student on train[:2%] of Wikitext-2-raw-v1 filtering by non-empty lines. For each batch, I compute teacher_logits = BASE(input_ids) and student_logits = PATCHED(input_ids), employing a token-shifted KL divergence between teacher and student logits as the loss.

This method ensures that instead of becoming a better LM, the row patch is trained to imitate BASE logits. Due to CPU limitations, I ran 2 epochs over the 2% slice with a batch size of 2 and max_len 128 at around 200 steps per epoch (i.e 400 optimization steps total). Logging the loss at each step showed noisy yet clear decreases, with many steps dropping drastically into 5-30

range after initially starting from 500-700+.



Final Pass: How Well is the Single-Weight Patch?

Quantitative Results After Patching

Running the Wikitext-2 validation metrics again yields:

```
=== METRICS ON WIKITEXT-2 (SMALL SLICE) ===  
BASE:      NLL=2.8572  PPL=17.41  KL(base||base)=0  
BROKEN:    NLL=7.9670  PPL=2884.33  KL(base||broken)=5.0269  
PATCHED:  NLL=3.2284  PPL=25.24  KL(base||patched)=0.3663
```

To analyze above numbers, I define the damage and residual as

$$\Delta_{\text{ablate}} = NLL_{\text{broken}} - NLL_{\text{base}} = 5.11 \text{ and } \Delta_{\text{residual}} = NLL_{\text{patched}} - NLL_{\text{base}} = 0.37$$

Therefore, the fraction of damage repaired is

$$\frac{\Delta_{\text{ablate}} - \Delta_{\text{residual}}}{\Delta_{\text{ablate}}} \approx 93\%.$$

Similarly, for KL instead of NLL I obtained a rounded result of 93%.

Behavioural Comparison: BASE vs BROKEN vs PATCHED

On the same hand-crafted prompt dataset designed to probe simple circuits, PATCHED looks much closer to BASE than to BROKEN, as seen below.

Example outputs (somewhat truncated)

a) "Summer is hot. Winter is"

- i) BASE: Summer is hot. Winter is cold.
- ii) BROKEN: Summer is hot. Winter is a good thing. I'd like to be good good good...
- iii) PATCHED: Summer is hot. Winter is cold. It is summer. It is hot. Winter is cold...

This is the correct answer with some added repetition.

b) "Paris is in France. Tokyo is in"

- i) BASE: ... Japan
- ii) BROKEN: ... the ocean. The ocean. The ocean. the. the. the. the. the.
- iii) PATCHED: Paris is in France. Tokyo is in Japan. Answer: Tokyo is in Japan.

This is the correct answer being more explicit than BASE.

c) "2, 4, 6, 8"

- i) BASE: 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28,
- ii) BROKEN: 2, 4, 6, 8, ', ', ', ', mar, mar, mar, mar, mar, mar, mar,
- iii) PATCHED: 2, 4, 6, 8, 10, 12, 16, 20, 24, 28, 32, 36, 40, 44,

This is mostly correct, incorrect jumps yet no catastrophic collapse.

d) "A cat is an animal. A rose is a"

- i) BASE: ... flower.
- ii) BROKEN: ... A. A. A. A...
- iii) PATCHED: A cat is an animal. A rose is a flower. A dog is an animal. A cat is an animal...

This is semantically correct, and also structurally sane.

```
=====
PROMPT: 'Summer is hot. Winter is'
BASE : Summer is hot. Winter is cold.

Question:
Die Kommission hat sich in der Vergangenheit mit der
BROKEN : Summer is hot. Winter is a good thing. I'd like to be good good good good good good good good good
PATCHED: Summer is hot. Winter is cold. It is summer. It is hot. Winter is cold. It is summer. It is
=====
PROMPT: 'Paris is in France. Tokyo is in'
BASE : Paris is in France. Tokyo is in Japan.

Question:
Die Kommission hat sich in der Vergangenheit mit der
BROKEN : Paris is in France. Tokyo is in the ocean. The ocean. The ocean. the. the. the. the. the.

PATCHED: Paris is in France. Tokyo is in Japan.
Answer:
Tokyo is in Japan.
=====
PROMPT: '2, 4, 6, 8,'
BASE : 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28,
BROKEN : 2, 4, 6, 8, ', ', ', ',
mar, mar, mar, mar, mar, mar, mar
PATCHED: 2, 4, 6, 8, 10, 12, 16, 20, 24, 28, 32, 36, 40, 44,
=====
PROMPT: 'The capital of Canada is'
BASE : The capital of Canada is Ottawa.

Question:
Die Kommission hat sich mit dem Thema der Klimaänder
BROKEN : The capital of Canada is a lot of people.
leaves of of of of of of of of of of
PATCHED: The capital of Canada is Ottawa. The largest city is Toronto. The country is also made up of many small provinces and territories
=====
```

Clearly, across example prompts, PATCHED appears to reproduce the base answer (or a close paraphrase) before drifting into mild verbosity or repetition. In contrast, BROKEN frequently fails

outright. Therefore, these outputs present strong behaviour evidence that the row-level patch recovers most of the capabilities that were lost by ablating the superweight.

What Does the Learned Patch Look Like?

Row-Level Geometry

From patch.safetensors in addition to the original states, I extract the norms:

- $\|base_row\| \approx 1.14$
- $\|broken_row\| \approx 0.95$
- $\|\Delta_{row}\| \approx 3.01$
- $\|merged_row\| \approx 3.27$
- $\|base_row - merged_row\| \approx 3.04$

On top of this, I gather the cosine similarities:

- $\cos(BASE, BROKEN) \approx 0.83$
- $\cos(BASE, DELTA) \approx 0.13$
- $\cos(BASE, MERGED) \approx 0.36$

As such, the ablated row is still largely aligned with the base row (i.e $\cos$ is approximately 0.83) because we only removed one scalar. In comparison, the Δ_{row} is nearly orthogonal to the base row, and the merged row (i.e $broken_row + \Delta_{row}$) points in a fairly different direction from the original row with a much larger norm.

This implies the student did not simply place back the missing scalar. Instead, it learned a different direction which helps it match BASE's logits.

Top-k Entries and Sparsity

Inspecting the top 20 entries in Δ_{row} I found that many spread ± 0.13 - 0.25 across the input dimension. Comparing the original superweight column at index 1710 I notice delta around $+0.163$ and base around $+0.63$, with the squared contribution being a tiny fraction of the total squared norm of Δ_{row} (i.e 0.3%). To get a better view, I ran a simple sparsity ablation for k

having values 8, 16, 32, and 64 by doing the following- I construct Δ_{row}^k by keeping only top- k entries (by $|\text{delta}|$) and zeroing everything else and then merging with BROKEN, sampling on the same prompts.

I'll now describe the qualitative observations of doing the above actions: For Top-8 I noted many prompts partially repaired but unusual errors like "Tokyo is in the United States" or odd quote behaviours were rampant, meaning worse than the full patch. In Top-16, I got better results yet it was very clearly degraded compared to full patch with some repetitions and large factual mistakes. On Top-32 and Top-64, example prompts like "Tokyo is in Japan", "The capital of Canada is Ottawa", lists, and "A rose is a flower" are mostly correct again, with only stylistic differences and mild repetitions.

```

--- TOP-16 SPARSE PATCH ---
=====
PROMPT : 'Summer is hot. Winter is'
OUTPUT : Summer is hot. Winter is cold.
Summer is hot. Winter is cold. Summer is hot. Winter is cold. Summer
=====
PROMPT : 'Paris is in France. Tokyo is in'
OUTPUT : Paris is in France. Tokyo is in the same place. Paris is in France France. Paris is in France France. Paris is in France
=====
PROMPT : '2, 4, 6, 8,'
OUTPUT : 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 24, 28, 30, 32,
=====
PROMPT : 'The capital of Canada is'
OUTPUT : The capital of Canada is Ottawa.
its capital of Canada.
Ottawa is the capital of Canada.
its capital
=====
PROMPT : 'He opened the quote " and then wrote Hello. To close it he should type'
OUTPUT : He opened the quote " and then wrote Hello. To close it he should type it. To close it he should type it. to close it. to close it. to close
=====
PROMPT : 'List of numers: [1, 2, 3'
OUTPUT : List of numers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 10, 10
=====
PROMPT : "In Python, to close a string started with ' you end with "
OUTPUT : In Python, to close a string started with'you end with
it is not a problem.

\begin{string}

\begin{ string
=====

--- TOP-64 SPARSE PATCH ---
=====
PROMPT : 'Summer is hot. Winter is'
OUTPUT : Summer is hot. Winter is cold.
Summer is hot. Winter is cold. Summer is hot. Winter is cold. Summer
=====
PROMPT : 'Paris is in France. Tokyo is in'
OUTPUT : Paris is in France. Tokyo is in Japan.
commons.
The Paris is in France. Paris is in France. Tokyo is
=====
PROMPT : '2, 4, 6, 8,'
OUTPUT : 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 24, 28, 32, 36,
=====
PROMPT : 'The capital of Canada is'
OUTPUT : The capital of Canada is Ottawa.
its capital is Ottawa.
its capital is Ottawa.
its capital is Ottawa.
=====
PROMPT : 'He opened the quote " and then wrote Hello. To close it he should type'
OUTPUT : He opened the quote " and then wrote Hello. To close it he should type Hello. To close it he should type Hello. To close it he should type Hello. To close
=====
PROMPT : 'List of numers: [1, 2, 3'
OUTPUT : List of numers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13
=====
PROMPT : "In Python, to close a string started with ' you end with "
OUTPUT : In Python, to close a string started with'you end with
it is a string.
=====

```

Reflecting on this behaviour suggests that the effective “circuit” realized by the patch uses on the order of tens of non-zero entries, not just one.

Influence Mapping

Separately, I computed per-token KL(BASE||BROKEN) across a Wikitext-2 slice and looked at the highest-scoring positions. The data showed that many of the top contexts clustered around marine biology + geography content (i.e European lobster, North/Black Sea, American lobster, coastal regions). This intriguingly lines up with the out-of-place “mar”, “maritimes”, “marina” behaviour I saw in BROKEN completions earlier. I did not fully develop this into a clean story, but my tentative hypothesis is that this superweight participates in a direction that is useful for certain “marine/coastal” and structural concepts, and, once removed, other directions become miscalibrated and show up as junk tokens (such as repeated “mar”). The row-level patch seems to reintroduce a more stable combination of those features.

```

=====
Score: 14.4995 Token: ' American'
Context: ' the Black Sea. It is closely related to the American lobster, H. americanus. It'
=====
Score: 14.0165 Token: ' zo'
Context: ' they drift with the ocean currents, preying on zooplankton. This stage involves three moul'
=====
Score: 13.9922 Token: ' pag'
Context: ' other European species such as the edible crab, Cancer pagurus. Between 1904 and 1914, one million'
=====
Score: 13.7511 Token: ' g'
Context: 'um Cycliophora ; the species on H. gammarus has not been described. \n'
=====
Score: 13.6393 Token: 'verte'
Context: 'ammarus mostly consists of other benthic invertebrates. These include crabs, mollus'
=====
Score: 13.5777 Token: ' North'
Context: 'chelde were distinct from those collected in the North Sea or English Channel. \n'
=====
Score: 13.5397 Token: 'ace'
Context: ' @. @ 9 lb ). Like other crustaceans, lobsters have a hard exos'
=====
Score: 13.1193 Token: ' English'
Context: ' were distinct from those collected in the North Sea or English Channel. \n'
=====
Score: 13.0400 Token: ' bur'
Context: ' although they are known to be capable of digging extensive burrows. It is estimated that only 1 lar'
=====
Score: 13.0390 Token: 'j'
Context: ' The northernmost populations are found in the Norwegian fjords Tysfjorden and Nordfold'
=====
Score: 12.9337 Token: 'os'
Context: ' crustaceans, lobsters have a hard exoskeleton which they must shed in order to grow'
=====
Score: 12.8881 Token: ' americ'
Context: ' closely related to the American lobster, H. americanus. It may grow to a length of'
=====
Score: 12.8773 Token: 'g'
Context: ' H. americanus and Nephrops norvegicus are hosts to the three known species of'
=====

```

Limitations

There are many limitations to this research on mechanistic interpretability. Firstly, I studied only one model, a single layer in said model, and one superweight featured in the model with one row patch. Due to strict CPU limits, training was quite limited so I used small dataset slices and could not perform hyperparameter searches. I also did not do deep mechanistic analysis of what those patched columns represent (i.e via activation patching, probes, or maximum-activating contexts). Additionally, the qualitative evaluation is a small prompt suite whereas a more thorough evaluation would require a larger and more systematic set of tasks. I attempted to be explicit about these limitations in the write-up and not over-claim. In full honesty, I view this as a proof-of-concept that row-level “circuit regrowth” around a superweight is feasible and interesting, not as a definitive characterization.

Possible Extensions + Next Steps

If I had more time (and compute), I would love to extend the project in a multitude of ways:

1. Extend to include multiple superweights and layers: I would repeat the ablation, patching and analysis for the second reported superweight in OLMo-1B as well as a superweight

in a different layer (i.e layer 0 or 2) to test whether my 93% repair on a single row is typical or special for my chosen weight.

2. Broaden for better mechanistic understanding: I would utilize activation patching to see which prompts and positions are most affected by the patched row, and train simple linear probes or inspect the maximum-activating contexts for the largest columns of Δ_{row} to see what features they correlate with (i.e marine-related tokens, quotes, list punctuation)
3. Systematic behaviour evaluation pipeline: I would construct a larger synthetic test set for tasks like quote closing, list continuation and factual Q&A, measuring the rate of accuracy + mistakes across BASE, BROKEN, and PATCHED to quantify how much structural behaviour is recovered.
4. Innovate concept editing angle: I would treat each row patch as a simple editing primitive by trying to train patches that explicitly suppress or amplify certain concept clusters (i.e marine/coastal tokens) while preserving performance elsewhere, mainly seeing if this can be used as a cheap “concept gain knob” compared to heavier tools like LoRA.
5. Compare to analytic patches: I have already verified that an analytic path (i.e putting back the original scalar at [1764, 1710]) exactly restores BASE behaviour, so I am interested in comparing that analytic patch, the learned row patch and a redistributed patch (i.e spread-k) side-by-side on both metrics and behaviours to better understand trade-offs between “reconcentrated” and “distributed” repairs